

Les ensembles implémentés via une table de *hashing*

Exercices obligatoires

A. Animation

Assurez-vous d'avoir bien compris ce qu'est une table de *hashing* via l'animation CASIER_HUMAIN que vous trouverez sur Moovin. Téléchargez le contenu du dossier "Simulation Table de Hachage" et décompressez les fichiers dans un dossier de votre choix. Vous y trouverez un guide détaillé. La Partie 1 de ce document vous permettra, si vous le souhaitez, de reproduire en autonomie la première partie de l'activité réalisée précédemment avec l'enseignant.

Exercice à réaliser : L'exercice consiste à compléter la Partie 2 du guide ("Exercice de synthèse"). Testez différentes fonctions, variez le paramètre k (avec les boutons + et -), et générez les tables pour relever les statistiques.

L'objectif est de trouver la combinaison qui offre la meilleure répartition possible.

Remplissez le tableau de synthèse remis avec vos observations.

B.Implémentation de l'interface *Ensemble* via une table de hashing

B1 Implémentez l'interface *Ensemble* en utilisant un tableau de listes.

Complétez la classe *EnsembleTableHashing*.

La classe *TestEnsembleTableHashing* permet de tester cette classe.

Utilisez la classe *ListeSimpleImp* que vous avez écrite ou la solution proposée sur moodle.

B2 Sur moodle, répondez au questionnaire à choix multiples *EnsembleTableHashing*.

C Application Parking

Le personnel de la société X qui compte 256 employés a le droit de placer sa voiture dans le parking de la société.

Pour éviter les indésirables, l'entrée du parking est munie d'une barrière.

La société décide d'équiper la barrière d'un détecteur de plaques. Celle-ci ne s'ouvrira que pour les plaques autorisées.

C1 Complétez la classe *Parking*.

Cette application *possède* les fonctionnalités suivantes :

```
boolean ajouterVoiture(Voiture voiture)
boolean voitureAutorisee(Voiture voiture)
boolean retirerVoiture(Voiture voiture)
String[] tableTriePlaques()
```

L'ensemble implémenté via une table de *hashing* est la structure de données la plus adéquate pour conserver les voitures autorisées.

Si la méthode `hashCode()` est bonne, les 3 premières méthodes sont en $O(1)$!

La classe *Voiture* possède une méthode `hashCode()` qui sera testée à l'exercice D2.

Attention, c'est cette méthode `hashCode()` qui est utilisée par la classe de tests.

Testez la classe avec la classe *TestParking*. Elle doit être en partie complétée.

C2 Sur moodle, répondez au questionnaire à choix multiples *Parking*.

D Méthode de *hashing* de la classe *Voiture*

Chaque objet de la classe *Voiture* possède une plaque de type :



D1 En vue de réaliser un graphique qui permet de voir comment la méthode `hashCode()` de la classe *Voiture* répartit les 17576000 voitures parmi 500 listes, complétez la classe *TestPlaqueDeVoiture*.

Pour ce faire, la classe va simuler toutes les voitures portant les numéros de plaque 1 chiffre-3 lettres-3 chiffres.

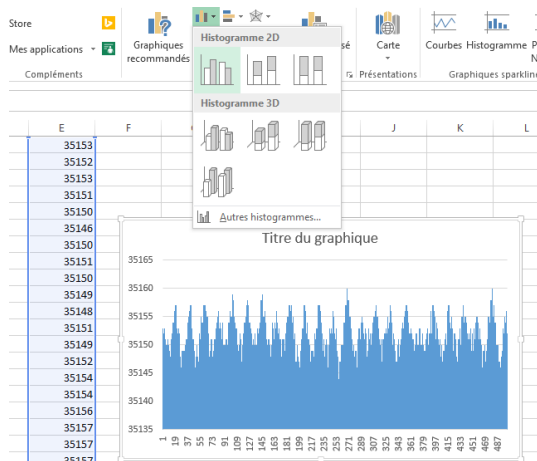
Pour chaque plaque simulée, le programme appelle la méthode `hashCode()` et déduit donc le numéro de la liste où placer la plaque.

Après toutes ces simulations, la classe va afficher à l'écran le nombre de plaques obtenues dans chaque liste. Les résultats vont correspondre aux ordonnées du graphique.

Il suffira de reporter (copier-coller) les résultats qui s'affichent à l'écran sur une page Excel et d'insérer un graphique type « Histogramme » avec ces résultats.

Remarque : il n'est pas nécessaire de créer 500 listes, 500 compteurs feront l'affaire ! Réfléchissez ...

Voici la démarche à suivre pour obtenir le graphique sous Excel :



D2

Commencez par sélectionner les données avant de choisir, dans le menu « insertion », le type de graphique histogramme

Remarque importante :

La classe `Object` propose une méthode `hashCode()`.

C'est celle-ci que vous redéfinissez ! Mais que se passe-t-il si vous ne le faites pas ?

Voici un exemple :

```

1  public class TestHashObject {
2
3
4  public static void main (String args[]) {
5      Voiture voiture1 = new Voiture("1AAA000", "p");
6      Voiture voiture2 = new Voiture("1AAA000", "p");
7      System.out.println(voiture1.hashCode());
8      System.out.println(voiture2.hashCode());
9  }
10
11 }
12

```

Console

<terminated> TestHashObject [Java Application] C:\Program Files (x86)\Java\jre1.8.0
27134973
1284693

Si la méthode `hashCode()` n'est pas redéfinie, elle renvoie l'adresse de l'objet. Dans l'exemple, nous avons 2 instances donc 2 adresses différentes.

Vous allez comparer, à l'aide de graphiques (en utilisant la classe `TestPlaqueDeVoiture`), différentes méthodes `hashCode()` de la classe `Voiture` :

a) La classe `String` a redéfini la méthode `hashCode()`.

La méthode ne renvoie pas l'adresse. Et heureusement l'entier renvoyé est le même pour 2 mêmes chaînes de caractères.

Nous pouvons peut-être l'utiliser ?

C'est-ce qui est fait dans la méthode `hashCode()` proposée par défaut.

Testez-là !

b) Testez la méthode proposée :

```
public int hashCode(){
    int resultat = 0;
    for(int i = 0 ; i < 7 ; i++){
        resultat += (int) this.numPlaque.charAt(i);
    }
    return resultat;
}
```

c) IntelliJ propose de générer automatiquement les méthodes `hashCode()` et `equals()`.

Faites-le !

Mettez bien en commentaires les méthodes `hashCode()` et `equals()` existantes.

Observez les méthodes générées.

Mise en garde :

C'est bien pratique, mais il ne faut surtout pas oublier de le faire !!!



d) Proposez une méthode `hashCode()` « idéale » pour les plaques 1 - 3 lettres – 3 chiffres en vue de répartir les plaques parmi 500 listes.

(N'en essayez pas plusieurs, réfléchissez plutôt à la *String* manipulée !)

Exercice défi



B3 Ajouter un attribut `loadFactor` et le constructeur `EnsembleTableHashing(int capacite, double loadFactor)` à votre classe *EnsembleTableHashing*.

Revoyez la méthode `ajouter()`.

Prévoyez un agrandissement de table dans le cas où $\text{taille/capacite} > \text{loadFactor}$.

Vous pouvez tester le défi en utilisant la classe *TestDefiEnsembleTableHashing* :

Exercice supplémentaire



B4 Modifiez la méthode `toString()` de la classe *EnsembleTableHashing*.

Celle-ci doit renvoyer uniquement les éléments de l'ensemble sans faire apparaître la structure de données choisie pour l'implémentation.

Exercice pour les plus mordus de l'animation CASIER_HUMAIN

La Partie 3 du guide propose une activité sur les tables de hachage de niveau avancé. Vous pourrez l'effectuer à titre optionnel **une fois que vous aurez terminé la fiche d'exercices proposée pour cette semaine.**

Cfr. guide pratique.